

Requirements for the Gridbot App (CryptoCosmo)

Background

The **CryptoCosmo gridbot** is a Python project on GitHub that implements a conservative grid-trading bot for cryptocurrency pairs. It provides two modes:

- **Simulation** – runs a random-walk price feed with an in-memory exchange for testing strategies.
- **Sandbox** – uses the `ccxt` library against the Binance testnet for simulated trading. API keys are loaded from environment variables and trading orders can be placed when `exchange.dry_run` is disabled.

The bot includes a web dashboard (`FastAPI` + `uvicorn`) with start/stop controls, symbol selection, and controls for grid parameters. Key components are `GridTrader`, `RiskGuard`, `PerformanceAnalyst`, `HealthMonitor`, `ProfitSweeper`, and persistence via SQLite. The `SentimentScout` is currently a stub that always returns a neutral sentiment. Overall, the project is well-structured but it lacks some of the engineering practices recommended for production-ready AI/LLM systems. The following requirements adapt the eight core components highlighted in the AI-engineering framework from the transcript ¹ ² to the gridbot application.

Disclaimer: This document is for educational purposes and should not be used as financial advice. Cryptocurrency trading carries risk. The gridbot is intended for simulation and sandbox use only; you should not use it to trade real funds without proper understanding and regulation compliance.

1 Problem framing and success metrics

Objective: Clearly define the problem the gridbot solves and establish measurable metrics.

- **Business problem** – The bot aims to manage a *grid trading strategy* that buys at evenly spaced price levels and sells when the price rises by one level. The goal is to earn incremental profit while controlling exposure and risk. Document the business objective in a README or project specification, explaining why grid trading is used and what success looks like.
- **Constraints** – Identify latency constraints (e.g., tick intervals), maximum capital exposure (`grid.max_exposure_quote`), and risk thresholds (`daily_loss_limit_pct`, `max_drawdown_pct`, and `crash_drop_pct`). These should be parameters in `config.yaml` with clear descriptions.
- **Success metrics** – The transcript stresses the need to define metrics at several levels ¹ :

- **User-facing metrics:** *net profit over a period*, *drawdown (%)*, *task completion rate* (e.g., hitting target profit), and *user satisfaction* if the bot is used by human traders.
- **Technical metrics:** *response latency* of the dashboard and trading loop, *order fill rate* (ratio of filled orders to total placed), and *error rates* (exceptions per tick).
- **System metrics:** *uptime*, *cost per request* (for API calls to the exchange), and *resource utilization* (CPU/memory). These metrics should be logged and exposed via the dashboard.
- **Documentation** – Record decisions and rationales in a `docs/requirements.md` file and update the README. This documentation will be useful for interviews or collaboration.

2 Configuration/prompt management and systematic tracking

The transcript recommends treating prompts as versioned components and systematically evaluating them ¹. Gridbot does not use LLM prompts, but the same principle applies to **configuration and trading logic**.

- **Separate configuration files** – Keep `config.yaml`, `.env`, and any strategy parameters separate from code. Provide example config files (`config.example.yaml`) with comments describing each field. This allows safe parameter tuning without modifying the code.
- **Version and track configs** – Use semantic versioning for configuration changes; commit config files with meaningful commit messages. When adjusting grid parameters (spacing, center, bounds), record the changes in a change-log so that outcomes can be traced back to specific configurations.
- **Test cases for grid logic** – Create a test suite that exercises `GridTrader.sync`, `RiskGuard.evaluate`, `PerformanceAnalyst.maybe_adjust`, and `HealthMonitor.healthy` with various market conditions. Use parameterized tests to cover different price sequences, volatility and drift values. When parameters change, run the test suite to ensure that risk limits and grid behaviour still hold.
- **Prompt/LLM extension (optional)** – If you later integrate an LLM (for example to summarize news or generate sentiment scores), treat each prompt version like a configuration: store prompts in their own files, evaluate them on a set of test questions, and use metrics (BLEU/ROUGE or LLM-as-judge) to compare versions ².

3 Model selection and evaluation

Although gridbot currently uses hand-crafted rules, it could benefit from machine-learning models in several areas (e.g., sentiment analysis, price forecasting). The framework stresses the importance of comparing models systematically rather than defaulting to the newest model ².

- **Sentiment analysis** – Replace the stub in `SentimentScout` with a real sentiment model. Options include:

- **Pre-built API** – call a news or social-media sentiment API (e.g., Fear & Greed Index or CryptoPanic) and parse the score.
- **LLM-based classification** – use a small LLM via OpenAI or a local model to classify current news headlines into positive/neutral/negative. Evaluate multiple models (different providers and sizes) on a labelled dataset (20–50 example headlines) and choose the model that offers the best balance between accuracy and API cost.
- **Price prediction/volatility models** – Experiment with simple statistical models (e.g., ARIMA, exponential moving averages) or LSTM/transformers to predict short-term volatility. Compare them against the existing random-walk simulation. For each model, measure prediction accuracy (e.g., RMSE), compute cost (compute time and memory), and assess whether the improvement justifies the added complexity.
- **Configuration search** – Use a grid-search or Bayesian optimization to find optimal parameters (spacing, number of levels, max exposure) given the simulated environment. Evaluate each parameter set on metrics defined above (profit, drawdown, risk) and record the results.
- **Documentation** – Document the model-selection process in the repository (e.g., `docs/model_selection.md`) and include charts showing performance vs. cost. Cite the reasoning behind choosing a particular model or leaving the stub if no model offers significant improvement.

4 Data retrieval and RAG (retrieval-augmented generation)

The transcript devotes a section to building retrieval-augmented generation (RAG) pipelines ². In gridbot, *retrieval* is still important even without text generation:

- **Market data retrieval** – Use the `ccxt` library to fetch current prices, order book depth and historical candles. Consider adding fallback APIs (e.g., CoinGecko, Coinbase) to handle data outages. Evaluate the accuracy and latency of each source and implement retries and caching.
- **News & sentiment retrieval** – Build a retrieval layer that scrapes or queries trusted news sources, Reddit/Discord sentiment, and on-chain indicators. Apply chunking strategies if you use LLMs to summarize long articles. Test different chunk sizes (e.g., 256 vs 512 tokens) and retrieval approaches (semantic vs keyword search) to see which provides the most relevant context for sentiment models. Track retrieval accuracy (how often relevant chunks are returned) and adjust chunking accordingly ².
- **Data storage** – Store retrieved data in a time-series database or SQLite tables separate from trading state. This enables offline analysis and debugging.
- **RAG evaluation (optional)** – If an LLM is used to synthesize insights from retrieved data (e.g., summarizing news for sentiment), create a test set of queries and expected outputs. Measure precision/recall of retrieved chunks and the accuracy of the generated summaries. Document these evaluations.

5 Agent systems

An agent in the AI-engineering context is an LLM that can call tools and take actions autonomously. Gridbot already implements a form of agent through `BotRunner`, which orchestrates components and interacts with the exchange. The following requirements align the bot with agent-system principles ²:

- **Modular tool functions** – Expose core actions (fetch current price, place/cancel order, adjust grid parameters, fetch sentiment) as explicit functions or methods. If you later integrate an LLM, use function-calling to let the LLM invoke these tools.
- **Error handling** – `BotRunner` and `GridTrader` already log exceptions, but more granular error types (network error, insufficient funds, API rate limit) should be categorized. Implement retry logic with exponential backoff and safe fallbacks.
- **Security** – Ensure that environment variables holding API keys are never logged. Provide role-based access control for the dashboard (e.g., an API key required to call `/api/start` and `/api/stop`). Use `.gitignore` to exclude `.env` files and add a `SECURITY.md` describing secret management practices.
- **Agent logging and monitoring** – Log every action the bot takes (placing orders, adjusting grid, seeding sells) with timestamps and outcomes. In the future, if an LLM agent is introduced, log the reasoning chain and decisions to debug misbehaviour.
- **Testing** – Create unit tests for each tool (e.g., `place_limit_buy`, `evaluate_risk`, `update_grid`), integration tests for the full trading loop, and adversarial tests (e.g., what if the price feed returns NaN or spikes unexpectedly?). Measure task completion rate and average steps, similar to the agent evaluation recommendations ².

6 Deployment and user interface

The transcript emphasises making the system accessible and reliable ². Gridbot already provides a dashboard; these requirements consolidate and extend deployment best practices.

- **REST API** – Continue serving the bot via FastAPI with endpoints for starting/stopping the bot (`/api/start`, `/api/stop`), updating grid parameters (`/api/grid`), and fetching state (`/api/state`). Document each endpoint with OpenAPI docs and include example requests/responses.
- **Streaming responses** – For user experience, consider server-sent events or WebSockets so that the dashboard updates in real-time instead of polling every two seconds. This reduces latency and bandwidth.
- **Authentication and rate limiting** – Protect API endpoints with an API key or OAuth token. Implement rate limiting to prevent abuse (e.g., no more than 10 start/stop requests per minute).

- **Deployment** – Provide Dockerfiles and deployment guides for local, testnet and production environments. Use environment variables for secrets and include a `Procfile` or Kubernetes manifest for cloud deployment.
- **User interface improvements** – The existing HTML dashboard is static and self-contained. Move styling to a separate CSS file and consider a frontend framework (React, Vue) if more complex interactions are planned. Provide feedback indicators (spinner, error messages) when actions fail.

7 System monitoring and error analysis

Monitoring separates a demo from a real system ². Gridbot includes a `HealthMonitor` and in-memory logging; further requirements enhance observability.

- **Centralized logging** – Persist logs to a file (`logs/gridbot.log`) or use a logging framework (e.g., `structlog`) that can send logs to a monitoring service like ELK or Grafana Loki. Include context fields: tick number, mode, symbol, action performed, and error details.
- **Metrics collection** – Export metrics such as tick latency, price feed latency, number of open orders, risk state (paused/active), drawdown, equity, error streaks, and sentiment scores. Expose these via a `/metrics` endpoint in Prometheus format.
- **Alerts and dashboards** – Set up alerting thresholds: if error streak ≥ 3 or price feed stale beyond `max_stale_price_secs`, send a notification (email or Slack). Provide dashboards showing equity over time, drawdown curves, profit sweeper balance, and grid parameter changes.
- **Post-mortem analysis** – When a run ends, save a report summarizing performance: total profit/loss, number of orders placed, number of pauses by `RiskGuard`, and health events. Use data from the persistence layer to generate charts. Document failure patterns and update test cases accordingly.

8 Fine-tuning and future improvements (optional)

Fine-tuning is less common than people think ², but it can be valuable if the bot integrates machine-learning models.

- **Model fine-tuning** – If you adopt a smaller LLM for sentiment analysis or decision-making, fine-tune it on domain-specific examples (crypto news labelled with sentiment, typical trading decisions) only after testing prompts thoroughly. Prepare a high-quality dataset (100+ examples) and evaluate the fine-tuned model against baseline prompts. Proceed only if the improvement justifies the additional cost.
- **Hyper-parameter optimization** – Fine-tune trading parameters (grid spacing, number of levels, risk limits) using simulation and cross-validation. Automate this process with a scheduler (e.g., `optuna`) that records experiments and results.

- **Extend sentiment and analytics** – Incorporate additional models (e.g., anomaly detection for price spikes, reinforcement learning to adapt grid parameters). Evaluate each addition systematically as described.

Conclusion and next steps

The gridbot project already implements many robust components (risk management, performance adjustment, health monitoring, persistence, UI). By following the AI-engineering framework's eight core components ¹ ², you can turn the bot into a production-ready system. Begin by documenting the problem and metrics, then gradually add configuration management, model evaluation, improved data retrieval, better agent orchestration, monitoring, and optional fine-tuning. Each change should be validated against the defined metrics to ensure that it solves a specific problem and improves the system rather than adding unnecessary complexity.

¹ ² Transcript of How to Build AI Engineering Projects That Get You Interviews -
YouTubeToTranscript.com

<https://youtubetotranscript.com/transcript>